Devolución detallada de la Entrega 1

Grupo G16

Nota de la Parte 1: 63.4 sobre 100

Compilación: Compilación confirmada mediante Maven con Java 25.

Resumen de criterios y puntajes

Criterio	Peso	Nivel	Puntaje
Nave y tripulación	18	Regular	10.8
Motor Warp y patrón State	8	Bien	6.4
Asistente y bitácora	10	Regular	6.0
Misiones y Template Method	10	Bien	8.0
Creación de naves y Factory	7	Excelente	7.0
Haberes y Decorator	7	Regular	4.2
Recursos e informe de misión	12	Regular	7.2
Contratos y tratamiento de errores	12	Regular	7.2
Verificación funcional	10	Insuficiente	3.0
Repositorio y documentación	6	Regular	3.6

Aspectos que requieren corrección o demostración

1 Nave y tripulación

Evaluación: Regular — 10.8 de 18 puntos.

Qué se observó: asignarTripulante agrega directamente a la lista estática Tripulante y getTripulante devuelve esa misma lista mutable. Las precondiciones escritas no se ejecutan.

Código involucrado: software/src/main/java/tpgProgra2026/Nave.java, líneas 36 a 57

```
36       * @post getTripulante().size() == tamanio anterior + 1
37       */
38      public void asignarTripulante(Tripulante t) {
39          Tripulante.add(t);
40      }
41
42      public boolean estaMotorDisponible() {
43          return motorwarp.estaMotorDisponible();
44      }
45
46      /**
47       * @pre  cantidad >= 0
48       * @post getRecursos().getCombustible() == combustible anterior + cantidad
49       */
50      public void cargarCombustible(int cantidad) {
51          recursos.setCombustible(cantidad);
52      }
53
54      /**
55       * @pre  cantidad >= 0
56       * @post getRecursos().getEnergia() == energia anterior + cantidad
57       */
```

Por qué requiere corrección: La colección compartida por todas las naves y expuesta al cliente permite que una nave modifique la tripulación de otra y evita controlar las reglas de incorporación.

Orientación para corregir: Hacer que la colección sea de instancia, privada y no modificable desde afuera; comprobar las precondiciones dentro de la operación.

2 Motor Warp y patrón State

Evaluación: Bien — 6.4 de 8 puntos.

Qué se observó: MotorWarp delega en objetos de estado, pero la entrega no deja igualmente visible, para todas las operaciones, qué transición inválida se rechaza y quién realiza ese rechazo.

Código involucrado: software/src/main/java/tpgProgra2026/MotorWarp.java, líneas 57 a 76

```
57
58    // transiciones
59    public void prepararSalto() {
60            estado.prepararSalto();
61    }
62
63    public void pasarAWarp() {
64            estado.pasarAWarp();
65    }
66
67    public void enfriar() {
68            estado.enfriar();
69    }
70
71    public void dejarDisponible() {
72            estado.dejarDisponible();
73    }
74
75  /*        public void abortar() {
76            estado.abortar();
```

Por qué requiere corrección: State no se verifica solo por la jerarquía. Debe observarse que el comportamiento cambia con el estado y que una operación no permitida no modifica silenciosamente el contexto.

Orientación para corregir: Completar o demostrar al menos una transición válida y una inválida por estado relevante. El estado debe decidir; MotorWarp solo conserva la referencia y delega.

3 Asistente y bitácora

Evaluación: Regular — 6.0 de 10 puntos.

Qué se observó: El asistente está presente, pero no se encontró una clase Bitacora que concentre una secuencia consultable de eventos. Archivos relacionados encontrados: Evento.

Código involucrado: software/src/main/java/tpgPrograma2026/Asistente.java, líneas 25 a 44

```
25       * @post bitacora.size() == tamanio anterior + 1
26       */
27      public void registrarEvento(String mensaje, String tipo) {
28          bitacora.add(new Evento(mensaje, tipo));
29      }
30
31      public boolean estaMotorDisponible() {
32          return getMotorWarp().estaMotorDisponible();
33      }
34
35      /**
36       * @pre  cantidad >= 0
37       * @post getRecursos().getCombustible() == combustible anterior + cantidad
38       */
39      public void cargarCombustible(int cantidad) {
40          getRecursos().setCombustible(cantidad);
41      }
42
43      /**
44       * @pre  cantidad >= 0
```

Por qué requiere corrección: Imprimir mensajes o crear eventos aislados no permite reconstruir después qué ocurrió, en qué orden y con qué resultado. La bitácora debe ser un objeto del dominio con una colección propia.

Orientación para corregir: Incorporar una Bitacora con operaciones de registro y consulta. Cada evento debe conservar como mínimo el tipo de operación y una descripción; si el diseño usa tiempo, también su marca temporal.

4 Misiones y Template Method

Evaluación: Bien — 8.0 de 10 puntos.

Qué se observó: La clase base reúne las etapas del ciclo, pero el método plantilla mostrado en Mision.java no está declarado final. Una subclase puede reemplazarlo y alterar el orden preparar–ejecutar–evaluar–cerrar.

Código involucrado: software/src/main/java/tpgProgra2026/Mision.java, líneas 1 a 24

```
   1  package tpgProgra2026;
   2
   3  /**
   4   * Clase abstracta que define una misión de la nave con el patrón Template Method: Ejecuto_Mision() fija
el orden de los pasos (preparar, ejecutar, EnergiaGanada, evaluar y cerrar) y las subclases aportan la energía
ganada y el identificador de la misión.
   5   *
   6   * @inv COSTO_COMBUSTIBLE y COSTO_DESGASTE son constantes (final) y estrictamente positivas.
   7   * @inv ac no cambia después de construida la misión.
   8   * @inv preparada es true solo si el último preparar() encontró recursos suficientes y el motor
disponible.
   9   */
  10  public abstract class Mision {
  11
  12      protected final int COSTO_COMBUSTIBLE = 4; // Por si luego cambia el valor
  13      protected final int COSTO_DESGASTE = 4;
  14      private boolean preparada = false;
  15      protected Asistente ac;
  16
  17      /**
  18       * @pre el Asistente ac recibido no es null y está inicializado.
  19       */
  20      public Mision(Asistente ac) {
  21          this.ac = ac;
  22      }
  23
  24      /**
```

Por qué requiere corrección: Template Method fija el esqueleto del algoritmo en la superclase. Las subclases deben completar variaciones puntuales, no sustituir el ciclo completo.

Orientación para corregir: Declarar inalterable el método plantilla y dejar como abstractos o hooks únicamente los pasos que realmente varían entre M01, M02 y M03.

5 Haberes y Decorator

Evaluación: Regular — 4.2 de 7 puntos.

Qué se observó: La jerarquía Decorator está definida en Decorator.java, pero no se encontró un escenario que envuelva un componente con dos adicionales consecutivos.

Código involucrado: software/src/main/java/tpgProgra2026/Decorator.java, líneas 5 a 23

```
   5  * @post getId() == tripulante.getId()
   6  */
   7  public abstract class Decorator extends Tripulante {
   8      protected Tripulante tripulante;
   9
  10      public Decorator( Tripulante tripulante ){
  11          super( tripulante.getAntiguedad() );
  12          this.tripulante = tripulante;
  13      }
  14      @Override
  15      //@post retorna el id del tripulante decorado
  16      public int getId(){
  17          return tripulante.getId();
  18      }
  19      // @post retorna un valor >= tripulante.getSueldo()
  20      public abstract double getSueldo();
  21      //@post retorna un String no nulo
  22      public abstract String descripcion();
  23  }
```

Por qué requiere corrección: Sin una composición real, la entrega demuestra herencia y delegación, pero no la ventaja central del patrón: combinar responsabilidades en tiempo de ejecución sin crear una subclase por combinación.

Orientación para corregir: Agregar un escenario de liquidación que parta del haber base y encadene al menos dos conceptos. Comprobar el monto final esperado sin trasladar el cálculo a Main.

6 Recursos e informe de misión

Evaluación: Regular — 7.2 de 12 puntos.

Qué se observó: Recursos.java contiene controles o actualizaciones de recursos, pero la entrega no demuestra que combustible, energía, desgaste e informe permanezcan coherentes ante éxito y aborto.

Código involucrado: software/src/main/java/tpgProgra2026/Recursos.java, líneas 13 a 36

```
13       * @post getCombustible() == combustible anterior + cantidad
14       */
15      public void setCombustible(int cantidad) {
16          combustible += cantidad;
17      }
18
19      /**
20       * @pre  cantidad >= 0
21       * @post getEnergia() == energia anterior + cantidad
22       */
23      public void setEnergia(int cantidad) {
24          energia += cantidad;
25      }
26
27      /**
28       * Reinicia el desgaste (mantenimiento).
29       * @post getDesgaste() == 0
30       */
31      public void setDesgaste() {
32          desgaste = 0;
33      }
34
35      /**
36       * @pre  combustible >= 0 && energia >= 0 && desgaste >= 0
```

Por qué requiere corrección: No alcanza con validar un valor aislado. La operación completa debe ser atómica desde el punto de vista del dominio: si la misión no puede realizarse, no debe dejar cambios parciales ni informar un resultado distinto del estado real.

Orientación para corregir: Agregar escenarios de éxito y rechazo que comparen valores antes y después. Generar el informe desde los recursos efectivamente consumidos, no desde constantes repetidas.

7 Contratos y tratamiento de errores

Evaluación: Regular — 7.2 de 12 puntos.

Qué se observó: En Nave.java las precondiciones aparecen escritas en comentarios, pero el método mostrado no las comprueba con contratos ejecutables.

Código involucrado: software/src/main/java/tpgProgra2026/Nave.java, líneas 1 a 22

```
 1  package tpgProgra2026;
 2  import java.util.ArrayList;
 3
 4  /**
 5   * @invariante id != null && nombre != null && tipo != null
 6   * @invariante recursos != null && motorwarp != null && Tripulante != null
 7   */
 8  public abstract class Nave {
 9      protected String id;
10      protected String nombre;
11      protected String tipo;
12      protected Recursos recursos;
13      protected MotorWarp motorwarp;
14      protected ArrayList<Tripulante> Tripulante = new ArrayList<>();
15
16      /**
17       * @pre  id != null && nombre != null && tipo != null && motorwarp != null
18       * @pre  combustible >= 0 && energia >= 0
19       * @post getRecursos().getCombustible() == combustible
20       * @post getRecursos().getEnergia() == energia
21       * @post getTripulante().isEmpty()
22       */
```

Por qué requiere corrección: Para la cátedra, una precondición es una obligación del invocante y se expresa con assert; no debe reemplazarse automáticamente por una excepción defensiva. Un comentario solo no detecta el uso incorrecto durante la verificación.

Orientación para corregir: Pasar las precondiciones centrales a assert y definir invariantes/postcondiciones donde correspondan. Reservar excepciones para situaciones del dominio que el programa deba gestionar.

8 Verificación funcional

Evaluación: Insuficiente — 3.0 de 10 puntos.

Qué se observó: No se encontró una clase ejecutable de verificación. El archivo mostrado es lo más cercano a una guía de uso, pero no permite comprobar automáticamente el comportamiento.

Código involucrado: README.md, líneas 1 a 6

```
1  # Trabajo Práctico Grupal de Programación C, FI, UNMdP, Argentina
2  Proyecto de Software embarcado de una nave interstelar tripulada.
3
4  - Código fuente (paquete con todas las clases) en la carpeta /software/src/main/java/tpgProgra2026/
5  - Informe en carpeta raíz /
6  - Consigna y demás documentos en la carpeta raíz /
```

Por qué requiere corrección: Sin un escenario ejecutable no puede reproducirse el efecto sobre estados, recursos, bitácora e informe.

Orientación para corregir: Agregar uno o más main breves que construyan el modelo, ejecuten casos válidos e inválidos y comprueben sus postcondiciones con assert o un verificador sencillo.

9 Repositorio y documentación

Evaluación: Regular — 3.6 de 6 puntos.

Qué se observó: README.md describe el proyecto, pero no proporciona una secuencia completa y verificable de comandos Maven para compilar y ejecutar los escenarios.

Código involucrado: README.md, líneas 1 a 6

```
1  # Trabajo Práctico Grupal de Programación C, FI, UNMdP, Argentina
2  Proyecto de Software embarcado de una nave interstelar tripulada.
3
4  - Código fuente (paquete con todas las clases) en la carpeta /software/src/main/java/tpgProgra2026/
5  - Informe en carpeta raíz /
6  - Consigna y demás documentos en la carpeta raíz /
```

Por qué requiere corrección: El repositorio debe poder ser usado por otra persona sin depender del IDE ni de conocimiento oral del grupo.

Orientación para corregir: Agregar requisitos, versión de Java, mvn clean test o package y el comando de ejecución de cada escenario principal.

Prioridades antes de la defensa

1. Implementar una bitácora consultable con eventos y orden temporal.
2. Completar el informe final y el control de recursos.
3. Agregar escenarios reproducibles con resultados esperados.